`interruptible_thread` constructor waits until `p` is no longer referenced by the new thread before returning. Note that this implementation doesn't take account of handling joining with the thread, or detaching it. You need to ensure that the `flag` variable is cleared when the thread exits, or is detached, to avoid a dangling pointer.

The `interrupt()` function is then relatively straightforward: if you have a valid pointer to an interrupt flag, you have a thread to interrupt, so you can just set the flag ❻. It's then up to the interrupted thread what it does with the interruption. Let's explore that next.

9.2.2. Detecting that a thread has been interrupted

You can now set the interruption flag, but that doesn't do you any good if the thread doesn't actually check whether it's being interrupted. In the simplest case you can do this with an `interruption_point()` function; you can call this function at a point where it's safe to be interrupted, and it throws a `thread_interrupted` exception if the flag is set:

```
void interruption_point()
{
    if(this_thread_interrupt_flag.is_set())
    {
        throw thread_interrupted();
    }
}
```

You can use such a function by calling it at convenient points within your code:

```
void foo()
{
    while(!done)
    {
        interruption_point();
        process_next_item();
    }
}
```